

UNINASSAU Centro Universitário Maurício de Nassau
CURSO DE CIÊNCIAS DA COMPUTAÇÃO / CÓDIGO DE ALTA
PERFORMANCE

Nome dos Alunos: JOÃO PEDRO MACHADO DA SILVA -
01782106/JORGE VILELA DOS SANTOS NETO - 01778458/
ANDRÉ GOMES MOURA - 01793413/ CARLOS VINÍCIUS
ANDRADE CARVALHOS - 01581860

SISTEMA DE CATÁLOGO DE LIVROS
INTEGRADO À API OPEN LIBRARY

Relatório Técnico

CIDADE - UF
2026

Sumário

1	Introdução	3
2	Tecnologias Utilizadas	3
3	Desenvolvimento do Backend	4
3.1	Ponto de Entrada (server.js)	4
3.2	Camada de Controle (bookController.js)	5
3.3	Camada de Serviços (bookService.js)	6
3.4	Mapeamento de Rotas (bookRoutes.js)	9
4	Infraestrutura e DevOps	9
4.1	Ambiente Isolado com Dockerfile	9
4.2	Dependências de Projeto (package.json)	10
5	Conclusão	10

1 Introdução

O acesso descentralizado e massivo a informações bibliográficas impõe o desafio de criar plataformas rápidas, eficientes e de fácil navegação para usuários finais. Este trabalho apresenta o desenvolvimento de um **Sistema de Catálogo de Livros**, composto por uma arquitetura cliente-servidor (*Full Stack*) integrada de forma dinâmica à API pública internacional *Open Library*.

O objetivo primordial do projeto é fornecer uma API intermediária capaz de abstrair as requisições complexas da biblioteca externa, higienizar e estruturar os dados, devolvendo para a interface uma resposta otimizada que inclui manipulação dinâmica de metadados, paginação parametrizada e caminhos diretos para renderização de capas de obras literárias.

2 Tecnologias Utilizadas

A engenharia do ecossistema baseia-se nas seguintes tecnologias:

- **Backend:** Node.js associado ao *framework* Express para roteamento HTTP assíncrono.
- **Frontend:** React.js para construção de uma interface de usuário baseada em componentes reativos (SPA).
- **Integração Externa:** Biblioteca Axios para o consumo de recursos via requisições baseadas em promessas (*Promises*).
- **Containerização:** Docker e imagens Alpine para padronização de ambiente de implantação e isolamento.

3 Desenvolvimento do Backend

O ecossistema do servidor foi modularizado utilizando o padrão arquitetural MVC (*Model-View-Controller*), segregando as responsabilidades de transporte de rotas, controle de requisição e regras de serviços especializados.

3.1 Ponto de Entrada (server.js)

O arquivo `server.js` inicializa o servidor Express, acopla middlewares globais essenciais (como segurança via CORS e interpretador de estruturas JSON) e disponibiliza as rotas da aplicação.

```
1 require('dotenv').config();
2 const express = require('express');
3 const cors = require('cors');
4 const bookRoutes = require('./routes/bookRoutes');
5
6 const app = express();
7 const PORT = process.env.PORT || 5000;
8
9 // Middleware
10 app.use(cors());
11 app.use(express.json());
12
13 // Rotas
14 app.use('/api/books', bookRoutes);
15
16 // Rota de teste
17 app.get('/api/health', (req, res) => {
18   res.json({ status: 'Server is running' });
19 });
20
21 // Iniciar servidor
22 app.listen(PORT, () => {
23   console.log('Servidor rodando na porta ${PORT}');
24 });
```

Listing 1: Código fonte do arquivo Server.js

3.2 Camada de Controle (bookController.js)

Responsável por interceptar os parâmetros de entrada oriundos do cliente via protocolo HTTP, validar e invocar as regras de negócio da camada de serviços.

```
1 const bookService = require('../services/bookService');
2
3 class BookController {
4   async searchBooks(req, res) {
5     try {
6       const { query, limit = 10 } = req.query;
7
8       if (!query) {
9         return res.status(400).json({ error: 'Query e obrigatorio' });
10      }
11
12      const books = await bookService.searchBooks(query, parseInt(limit)
13    );
14
15      const booksWithCovers = books.map((book) => ({
16        ...book,
17        cover_url: book.cover_id
18          ? bookService.getCoverUrl(book.cover_id)
19          : null,
20      }));
21
22      res.json(booksWithCovers);
23    } catch (error) {
24      res.status(500).json({ error: error.message });
25    }
26  }
27  async searchByAuthor(req, res) {
28    try {
29      const { author, limit = 10 } = req.query;
30
31      if (!author) {
32        return res.status(400).json({ error: 'Parametro author e
33        obrigatorio' });
34      }
35
36      const books = await bookService.searchByAuthor(author, parseInt(
37        limit));
38
39      const booksWithCovers = books.map((book) => ({
40        ...book,
41        cover_url: book.cover_id
42          ? bookService.getCoverUrl(book.cover_id)
```

```

41         : null,
42     }));
43
44     res.json(booksWithCovers);
45 } catch (error) {
46     res.status(500).json({ error: error.message });
47 }
48 }
49
50 async getBookDetails(req, res) {
51     try {
52         const { bookId } = req.params;
53
54         const bookDetails = await bookService.getBookDetails(bookId);
55         bookDetails.cover_url = bookDetails.cover_id
56             ? bookService.getCoverUrl(bookDetails.cover_id, 'L')
57             : null;
58
59         res.json(bookDetails);
60     } catch (error) {
61         res.status(500).json({ error: error.message });
62     }
63 }
64 }
65
66 module.exports = new BookController();

```

Listing 2: Camada de Controle - bookController.js

3.3 Camada de Serviços (bookService.js)

Esta camada concentra a lógica de consumo de dados externos da API *Open Library*. Aqui é feita a desestruturação do JSON retornado e o tratamento para evitar valores nulos.

```

1 const axios = require('axios');
2
3 const OPEN_LIBRARY_API = 'https://openlibrary.org';
4
5 class BookService {
6     async searchBooks(query, limit = 10) {
7         try {
8             const response = await axios.get(`${OPEN_LIBRARY_API}/search.json
9             ', {
10                 params: {
11                     title: query,
12                     limit: limit,

```

```

12     },
13   });
14
15   return response.data.docs.map((book) => ({
16     id: book.key,
17     title: book.title,
18     author: book.author_name ? book.author_name[0] : 'Desconhecido',
19     year: book.first_publish_year || 'N/A',
20     isbn: book.isbn ? book.isbn[0] : null,
21     cover_id: book.cover_i || null,
22   }));
23 } catch (error) {
24   console.error('Erro ao buscar livros:', error.message);
25   throw new Error('Erro ao buscar livros na API externa');
26 }
27 }
28
29 async searchByAuthor(author, limit = 10) {
30   try {
31     const response = await axios.get(`${OPEN_LIBRARY_API}/search.json
32     ', {
33       params: {
34         author: author,
35         limit: limit,
36       },
37     });
38
39     return response.data.docs.map((book) => ({
40       id: book.key,
41       title: book.title,
42       author: book.author_name ? book.author_name[0] : 'Desconhecido',
43       year: book.first_publish_year || 'N/A',
44       isbn: book.isbn ? book.isbn[0] : null,
45       cover_id: book.cover_i || null,
46     }));
47   } catch (error) {
48     console.error('Erro ao buscar por autor:', error.message);
49     throw new Error('Erro ao buscar livros por autor');
50   }
51 }
52
53 async getBookDetails(bookId) {
54   try {
55     const path = bookId.startsWith('/') ? bookId : `/works/${bookId}`;
56     const response = await axios.get(`${OPEN_LIBRARY_API}${path}.json
57     ');

```

```

57     const book = response.data;
58     return {
59         id: bookId,
60         title: book.title,
61         author: book.authors
62             ? book.authors.map((a) => a.name || a.author?.name).join(', ')
63             : 'Desconhecido',
64         description:
65             book.description && typeof book.description === 'string'
66             ? book.description
67             : book.description?.value || 'Sem descricao',
68         pages: book.number_of_pages || book.pagination || 'N/A',
69         isbn: book.isbn_13 ? book.isbn_13[0] : book.isbn_10 ? book.
70         isbn_10[0] : null,
71         cover_id: book.covers ? book.covers[0] : null,
72     };
73 } catch (error) {
74     console.error('Erro ao obter detalhes do livro:', error.message);
75     throw new Error('Erro ao obter detalhes do livro');
76 }
77
78 getCoverUrl(coverId, size = 'M') {
79     if (!coverId) return null;
80     return `https://covers.openlibrary.org/b/id/${coverId}-${size}.jpg`;
81 }
82 }
83
84 module.exports = new BookService();

```

Listing 3: Camada de Integração - bookService.js

3.4 Mapeamento de Rotas (bookRoutes.js)

A camada de mapeamento encapsula os endpoints REST expondo os verbos HTTP para o cliente externo.

```
1 const express = require('express');
2 const bookController = require('../controllers/bookController');
3
4 const router = express.Router();
5
6 router.get('/search', (req, res) => bookController.searchBooks(req, res)
7   );
8 router.get('/author', (req, res) => bookController.searchByAuthor(req,
9   res));
10 router.get('/:bookId', (req, res) => bookController.getBookDetails(req,
11   res));
12
13 module.exports = router;
```

Listing 4: Mapeamento de Endpoints - bookRoutes.js

4 Infraestrutura e DevOps

Para garantir a portabilidade e facilitar o deploy em servidores de produção, a aplicação foi empacotada utilizando containers virtuais isolados.

4.1 Ambiente Isolado com Dockerfile

O uso da distribuição leve `node:18-alpine` garante uma imagem otimizada com consumo de recursos reduzido.

```
1 FROM node:18-alpine
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 RUN npm install
7
8 COPY . .
9
10 EXPOSE 5000
11 CMD ["npm", "start"]
```

Listing 5: Configuração de Containerização - Dockerfile

4.2 Dependências de Projeto (package.json)

Abaixo são mapeadas as bibliotecas em suas versões semânticas estáveis responsáveis pelo ecossistema do projeto.

```
1 {
2   "name": "book-catalog-api",
3   "version": "1.0.0",
4   "description": "API para Catalogo de Livros",
5   "main": "src/server.js",
6   "scripts": {
7     "start": "node src/server.js",
8     "dev": "nodemon src/server.js"
9   },
10  "keywords": ["books", "api", "rest"],
11  "author": "",
12  "license": "MIT",
13  "dependencies": {
14    "express": "^4.18.2",
15    "cors": "^2.8.5",
16    "dotenv": "^16.3.1",
17    "axios": "^1.6.2"
18  },
```

```
19  "devDependencies": {  
20    "nodemon": "^3.0.1"  
21  }  
22 }
```

Listing 6: Manifesto de Dependências do Node - package.json

5 Conclusão

O sistema desenvolvido atendeu com eficácia todos os requisitos propostos de engenharia de software acadêmica. A separação em camadas bem definidas no backend provou-se ideal para a manutenção e escalabilidade da aplicação. A integração com o *Open Library* demonstrou a viabilidade técnica de reutilização de repositórios globais públicos para enriquecimento de ecossistemas corporativos e educacionais.